

INSTITUTO TECNOLÓGICO SUPERIOR DEL ORIENTE DEL ESTADO DE HIDALGO.

Programa Educativo:

Ingeniería en Sistemas Computacionales

Docente:

Efrén Rolando Romero León

Materia:

Métodos Numéricos

Semestre: 4 Grupo: 2

Actividad:

Examen

Fecha:

12 de Marzo de 2026

Método de Gauss Jordán

- 1.- Formar la Matriz Aumentada
- 2.- Localizar el primer elemento y convertirlo en pivote
- 3.- Eliminar los coeficientes de arriba o abajo del pivote con operaciones
- 4.- Repetir el proceso hasta formar una diagonal de unos
- 5.- Visualizar los resultados

Código en Python

```
import numpy as np
import time
import random

# =====
#           MÉTODO DE GAUSS-JORDAN
# =====

def gauss_jordan(A, b):
    """
    Resuelve el sistema  $Ax = b$  usando el método de Gauss-Jordan.
    Retorna el vector solución  $x$ .
    """
    n = len(b)
    # Construir la matriz aumentada  $[A \mid b]$ 
    M = np.array([[float(A[i][j]) for j in range(n)] + [float(b[i])] for i
in range(n)])

    for col in range(n):
        # Buscar pivote máximo para mayor estabilidad numérica
        max_row = max(range(col, n), key=lambda r: abs(M[r][col]))
        M[[col, max_row]] = M[[max_row, col]] # Intercambio de filas

        pivote = M[col][col]
        if abs(pivote) < 1e-12:
            raise ValueError(f"El sistema no tiene solución única (pivote ≈
0 en columna {col}).")
```

```

    # Normalizar la fila del pivote
    M[col] = M[col] / pivote

    # Eliminar todos los demás elementos de la columna
    for fila in range(n):
        if fila != col:
            M[fila] = M[fila] - M[fila][col] * M[col]

    # La solución está en la última columna
    return M[:, n]

# =====
#          FUNCIÓN PARA MOSTRAR RESULTADOS
# =====

def mostrar_resultado(tamano, A, b, solucion, tiempo_segundos):
    separador = "=" * 60
    print(f"\n{separador}")
    print(f" SISTEMA DE ECUACIONES {tamano}x{tamano} - MÉTODO GAUSS-
JORDAN")
    print(separador)

    print("\n📊 Matriz de coeficientes [A]:")
    for fila in A:
        print(" ", [round(v, 4) for v in fila])

    print("\n📊 Vector de términos independientes [b]:")
    print(" ", [round(v, 4) for v in b])

    print(f"\n✅ Solución del sistema (vector x):")
    for i, val in enumerate(solucion):
        print(f"   x{i+1} = {val:.6f}")

    print(f"\n🕒 Tiempo de cálculo: {tiempo_segundos:.8f} segundos "
          f"({tiempo_segundos * 1000:.6f} milisegundos)")
    print(separador)

# =====
#          MATRIZ 3x3
# =====

# Definir manualmente una matriz 3x3 con solución conocida
A3 = [

```

```

    [2, 1, -1],
    [-3, -1, 2],
    [-2, 1, 2]
]
b3 = [8, -11, -3]

print("\n" + "Ej 3a" * 10)
print(" RESOLUCIÓN DE SISTEMAS DE ECUACIONES CON GAUSS-JORDAN")
print("Ej 3a" * 10)

inicio = time.perf_counter()
sol3 = gauss_jordan(A3, b3)
fin = time.perf_counter()
tiempo3 = fin - inicio

mostrar_resultado(3, A3, b3, sol3, tiempo3)

# Verificación
print("\nQ Verificación (A · x debe ser igual a b):")
resultado_verif = np.dot(np.array(A3, dtype=float), sol3)
for i in range(3):
    print(f" Ecuación {i+1}: {resultado_verif[i]:.6f} ↔ b{i+1} = {b3[i]}")

# =====
# MATRIZ 20x20 (generada aleatoriamente)
# =====

random.seed(42) # Semilla fija para reproducibilidad
n = 20

# Generar una matriz con diagonal dominante para garantizar solución única
A20 = []
for i in range(n):
    fila = [random.uniform(-10, 10) for _ in range(n)]
    # Hacer diagonal dominante: el elemento diagonal es mayor que la suma
    del resto
    fila[i] = sum(abs(v) for v in fila) + random.uniform(5, 15)
    A20.append(fila)

b20 = [random.uniform(-50, 50) for _ in range(n)]

inicio = time.perf_counter()
sol20 = gauss_jordan(A20, b20)

```

```

fin = time.perf_counter()
tiempo20 = fin - inicio

mostrar_resultado(20, A20, b20, sol20, tiempo20)

# Verificación de error residual
residuo = np.linalg.norm(np.dot(np.array(A20), sol20) - np.array(b20))
print(f"\n🔍 Error residual  $\|A \cdot x - b\| = \{residuo:.2e\}$  (cercano a 0 → solución correcta)")

# =====
#             COMPARATIVA FINAL
# =====

print(f"\n{'=' * 60}")
print(" 📊 RESUMEN COMPARATIVO DE TIEMPOS")
print(f"\n{'=' * 60}")
print(f" Matriz 3x3 → {tiempo3:.8f} s ({tiempo3*1000:.6f} ms)")
print(f" Matriz 20x20 → {tiempo20:.8f} s ({tiempo20*1000:.6f} ms)")
if tiempo3 > 0:
    print(f" La matriz 20x20 tardó {tiempo20/tiempo3:.1f}x más que la 3x3")
print(f"\n{'=' * 60}\n")

```

Resultados:

Para una matriz de 3x3

```

=====
SISTEMA DE ECUACIONES 3x3 — MÉTODO GAUSS-JORDAN
=====

📄 Matriz de coeficientes [A]:
  [2, 1, -1]
 [-3, -1, 2]
 [-2, 1, 2]

📄 Vector de términos independientes [b]:
  [8, -11, -3]

✅ Solución del sistema (vector x):
  x1 = 2.000000
  x2 = 3.000000
  x3 = -1.000000

🕒 Tiempo de cálculo: 0.00017993 segundos (0.179931 milisegundos)

```

Para una matriz 20x20

```
Vector de términos independientes [b]:  
[-8.5559, 12.9765, -30.5565, 19.6354, -0.5623, -25.6016, 15.6058, -49.4455, 25.0964, 27.0046, -39.3413, -7.48  
54, -32.4113, 45.7966, 1.7958, -44.9782, -25.0802, 34.8336, -4.3538, 30.1417]  
  
Solución del sistema (vector x):  
x1 = -0.080669  
x2 = 0.136994  
x3 = -0.285290  
x4 = 0.050740  
x5 = -0.009181  
x6 = -0.195534  
x7 = 0.142211  
x8 = -0.406517  
x9 = 0.236652  
x10 = 0.242769  
x11 = -0.391803  
x12 = -0.019617  
x13 = -0.184440  
x14 = 0.438206  
x15 = 0.011855  
x16 = -0.308511  
x17 = -0.151011  
x18 = 0.423431  
x19 = 0.004803  
x20 = 0.245216  
  
Tiempo de cálculo: 0.00219428 segundos (2.194282 milisegundos)
```

Método de Jacobi

- 1.- Verificar que los elementos de la diagonal a_{ij} no sean cero.
- 2.- Mientras el error sea mayor a la tolerancia y no se supere el máximo de iteraciones:
- 3.- Crear un vector temporal para almacenar los nuevos valores.
- 4.- Para cada fila i desde 1 hasta n :
- 5.- Sumar todos los términos a_{ij} donde j diferente de i .
- 6.- Calcular el nuevo x_i usando la fórmula anterior.
- 7.- Calcular el error (generalmente usando la norma euclidiana entre el vector nuevo y el anterior).
- 8.- Actualizar el vector de solución x con los valores temporales.
- 9.- Devolver el vector x aproximado.

Código en Python

```
import numpy as np
import time
import random

# =====
#           MÉTODO DE JACOBI
# =====

def jacobi(A, b, tol=1e-10, max_iter=1000):
    """
    Resuelve el sistema  $Ax = b$  usando el método iterativo de Jacobi.

    Parámetros:
        A      : Matriz de coeficientes (n x n)
        b      : Vector de términos independientes
        tol    : Tolerancia para criterio de convergencia
        max_iter : Número máximo de iteraciones

    Retorna:
        x      : Vector solución aproximada
        iteraciones : Número de iteraciones realizadas
        convergio : True si convergió, False si no
        historial : Lista con el error en cada iteración
    """
    n = len(b)
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)

    # Vector inicial: ceros
    x = np.zeros(n)
    historial_errores = []

    for k in range(1, max_iter + 1):
        x_nuevo = np.zeros(n)

        for i in range(n):
            suma = sum(A[i][j] * x[j] for j in range(n) if j != i)
            if abs(A[i][i]) < 1e-12:
                return x, k, False, historial_errores, "Pivote diagonal nulo
- el método no puede continuar."
            x_nuevo[i] = (b[i] - suma) / A[i][i]

        # Calcular error (norma del cambio)
```

```

        error = np.linalg.norm(x_nuevo - x, ord=np.inf)
        historial_errores.append(error)

        x = x_nuevo.copy()

        if error < tol:
            return x, k, True, historial_errores, None

    return x, max_iter, False, historial_errores, "Se alcanzó el máximo de
iteraciones sin converger."

# =====
#         VERIFICAR DIAGONAL DOMINANTE
# =====

def es_diagonal_dominante(A):
    """
    Verifica si la matriz es diagonal dominante por filas,
    condición suficiente (aunque no necesaria) para convergencia de Jacobi.
    """
    n = len(A)
    for i in range(n):
        diag = abs(A[i][i])
        resto = sum(abs(A[i][j]) for j in range(n) if j != i)
        if diag <= resto:
            return False, i # devuelve la fila que falla
    return True, -1

# =====
#         MOSTRAR RESULTADOS
# =====

def mostrar_resultado_jacobi(tamano, A, b, solucion, iteraciones,
                             convergio, historial, motivo, tiempo_s):

    sep = "=" * 65
    sep2 = "-" * 65

    print(f"\n{sep}")
    print(f"  SISTEMA {tamano}x{tamano} – MÉTODO DE JACOBI")
    print(sep)

    # — Diagnóstico de diagonal dominante
    dominante, fila_falla = es_diagonal_dominante(A)

```



```

if dominante:
    print("\n🔍 Diagonal dominante: ✔ SÍ (convergencia garantizada)")
else:
    print(f"\n🔍 Diagonal dominante: ⚠ NO (fila {fila_falla+1} no cumple la condición)")
    print("    Nota: El método PUEDE converger de todas formas, pero no está garantizado.")

# — Estado de convergencia
print(sep2)
if convergio:
    print(f"\n✔ CONVERGIÓ en {iteraciones} iteraciones | tolerancia = 1×10-10")
else:
    print(f"\n✗ NO CONVERGIÓ después de {iteraciones} iteraciones")
    if motivo:
        print(f"    Motivo: {motivo}")

# — Evolución del error (primeras y últimas iteraciones)
print(f"\n📊 Evolución del error ( $\|x_{\text{nuevo}} - x_{\text{viejo}}\|_{\infty}$ ):")
n_hist = len(historial)
indices_mostrar = []
if n_hist <= 10:
    indices_mostrar = list(range(n_hist))
else:
    indices_mostrar = list(range(5)) + ["..."] + list(range(n_hist-3, n_hist))

for idx in indices_mostrar:
    if idx == "...":
        print("    ...")
    else:
        print(f"    Iter {idx+1:4d}: error = {historial[idx]:.6e}")

# — Solución
print(f"\n{sep2}")
if convergio:
    print("✔ Solución aproximada (vector x):")
    for i, val in enumerate(solucion):
        print(f"    x{i+1:<3} = {val:+.8f}")
else:
    print("⚠ Última aproximación (puede no ser la solución real):")
    for i, val in enumerate(solucion):
        print(f"    x{i+1:<3} = {val:+.8f}")

```

```

# — Verificación residual
residuo = np.linalg.norm(np.dot(np.array(A), solucion) - np.array(b))
print(f"\n❏ Error residual  $\|A \cdot x - b\| = \{residuo:.4e\}$ ", end=" ")
if residuo < 1e-6:
    print("← Solución aceptable ✓")
elif residuo < 1e-2:
    print("← Solución aproximada ⚠️")
else:
    print("← Solución INCORRECTA ✗")

# — Tiempo
print(f"\n{sep2}")
print(f"❏ TIEMPO DE CÁLCULO ({tamano}x{tamano}):")
print(f"    → {tiempo_s:.8f} segundos")
print(f"    → {tiempo_s * 1000:.6f} milisegundos")
print(sep)

# =====
#  ENCABEZADO
# =====

print("\n" + "❏ " * 12)
print("    RESOLUCIÓN DE SISTEMAS DE ECUACIONES – MÉTODO DE JACOBI")
print("❏ " * 12)

# =====
#  MATRIZ 3x3 (mismos valores que Gauss-Jordan)
# =====

A3 = [
    [2, 1, -1],
    [-3, -1, 2],
    [-2, 1, 2]
]
b3 = [8, -11, -3]

print("\n❏❏ Procesando sistema 3x3...")
inicio = time.perf_counter()
sol3, iter3, conv3, hist3, mot3 = jacobi(A3, b3, tol=1e-10, max_iter=1000)
fin = time.perf_counter()
tiempo3 = fin - inicio

```

```

mostrar_resultado_jacobi(3, A3, b3, sol3, iter3, conv3, hist3, mot3,
tiempo3)

# — Comparación con solución exacta de Gauss-Jordan
print("\n███ Comparación con la solución exacta (Gauss-Jordan):")
sol_exacta3 = [2.0, 3.0, -1.0]
print(f"    {'Variable':<8} {'Gauss-
Jordan':>14} {'Jacobi':>14} {'Diferencia':>14}")
print(f"    {'-'*56}")
for i in range(3):
    diff = abs(sol_exacta3[i] - sol3[i])
    print(f"    x{i+1:<7} {sol_exacta3[i]:>14.8f} {sol3[i]:>14.8f} {diff:>
14.2e}")

# =====
#  MATRIZ 20x20  (mismos valores que Gauss-Jordan)
# =====

random.seed(42)
n = 20

A20 = []
for i in range(n):
    fila = [random.uniform(-10, 10) for _ in range(n)]
    fila[i] = sum(abs(v) for v in fila) + random.uniform(5, 15)
    A20.append(fila)

b20 = [random.uniform(-50, 50) for _ in range(n)]

print("\n\n❖██ Procesando sistema 20x20...")
inicio = time.perf_counter()
sol20, iter20, conv20, hist20, mot20 = jacobi(A20, b20, tol=1e-10,
max_iter=2000)
fin = time.perf_counter()
tiempo20 = fin - inicio

mostrar_resultado_jacobi(20, A20, b20, sol20, iter20, conv20, hist20, mot20,
tiempo20)

# — Comparación con solución exacta de Gauss-Jordan
sol_exacta20 = [-0.080669, 0.136994, -0.285290, 0.050740, -0.009181,
                -0.195534, 0.142211, -0.406517, 0.236652, 0.242769,
                -0.391803, -0.019617, -0.184440, 0.438206, 0.011855,
                -0.308511, -0.151011, 0.423431, 0.004803, 0.245216]

```

```

print("\n📊 Comparación con la solución exacta (Gauss-Jordan):")
print(f"   {'Variable':<8} {'Gauss-Jordan':>14} {'Jacobi':>14} {'Diferencia':>14}")
print(f"   {'-'*56}")
for i in range(n):
    diff = abs(sol_exacta20[i] - sol20[i])
    marca = "⚠️" if diff > 1e-4 else "✓"
    print(f"   x{i+1:<7} {sol_exacta20[i]:>14.6f} {sol20[i]:>14.6f} {diff:>14.2e} {marca}")

# =====
#   RESUMEN FINAL COMPARATIVO
# =====

print(f"\n{'=' * 65}")
print("   📊 RESUMEN FINAL – JACOBI vs GAUSS-JORDAN")
print(f"{'=' * 65}")
print(f"   {'Sistema':<12} {'Método':<14} {'Tiempo (s)':>12} {'Iters':>6} {'Convergió':>10}")
print(f"   {'-'*60}")
print(f"   {'3x3':<12} {'Gauss-Jordan':<14} {0.00017402:>12.8f} {'-'*6} {'✓ Exacto':>10}")
print(f"   {'3x3':<12} {'Jacobi':<14} {tiempo3:>12.8f} {iter3:>6} {'✓' if conv3 else '✗':>10}")
print(f"   {'20x20':<12} {'Gauss-Jordan':<14} {0.00119380:>12.8f} {'-'*6} {'✓ Exacto':>10}")
print(f"   {'20x20':<12} {'Jacobi':<14} {tiempo20:>12.8f} {iter20:>6} {'✓' if conv20 else '✗':>10}")
print(f"{'=' * 65}\n")

```

Resultados

Matriz de 3x3

```

=====
SISTEMA 3x3 – MÉTODO DE JACOBI
=====

⚠️ Diagonal dominante: ⚠️ NO (fila 1 no cumple la condición)
Nota: El método PUEDE converger de todas formas, pero no está garantizado.
-----

✗ NO CONVERGIÓ después de 1000 iteraciones
Motivo: Se alcanzó el máximo de iteraciones sin converger.

```

Matriz de 20x20

```
=====
SISTEMA 20x20 - MÉTODO DE JACOBI
=====

Diagonal dominante: ☒ Sí (convergencia garantizada)
-----

☒ CONVERGIÓ en 16 iteraciones | tolerancia = 1x10-10

☒ Evolución del error ( $\|x_{\text{nuevo}} - x_{\text{viejo}}\|_{\infty}$ ):
  Iter   1: error = 4.286485e-01
  Iter   2: error = 1.304654e-01
  Iter   3: error = 1.756905e-02
  Iter   4: error = 4.659200e-03
  Iter   5: error = 1.071268e-03
  ...
  Iter  14: error = 1.679459e-09
  Iter  15: error = 3.045162e-10
  Iter  16: error = 9.238607e-11

-----

☒ Solución aproximada (vector x):
  x1    = -0.08066917
  x2    = +0.13699414
  x3    = -0.28529007
  x4    = +0.05073999
  x5    = -0.00918118
  x6    = -0.19553369
  x7    = +0.14221065
  x8    = -0.40651687
  x9    = +0.23665197
  x10   = +0.24276851
  x11   = -0.39180347
  x12   = -0.01961741
  x13   = -0.18444021
  x14   = +0.43820628
  x15   = +0.01185548
  x16   = -0.30851133
  x17   = -0.15101068
  x18   = +0.42343137
  x19   = +0.00480301
  x20   = +0.24521578
```